Kotlin 编程入门

"Any sufficiently advanced app is indistinguishable from magic."

张丽，井明
数据科学与计算机学院
jingming@sdu.edu.cn
2026年秋

山东女子学院
Shandong Women's University

目录

CONTENTS

1 Kotlin 简介

2 基本语法

3 数据类型

4 控制语句

5 函数与Lambda

6 面向对象编程

目录

CONTENTS

7 集合与泛型

8 空安全机制

9 扩展与高阶函数

10 协程

11 Kotlin 与 Java 互操作

12 实践案例

第一章 Kotlin 简介

- Kotlin 是 JetBrains 开发的现代编程语言
- 2011年首次发布，2017年成为 Android 官方语言
- 设计目标：简洁、安全、互操作、工具友好

Kotlin 特点

- 与 Java 完全互操作
- 语法简洁，减少样板代码
- 内置空安全机制
- 支持函数式编程
- 强大的协程支持

Kotlin 应用场景

- Android 应用开发
- 后端开发（Ktor, Spring Boot）
- Web 前端（Kotlin/JS）
- 跨平台开发（Kotlin Multiplatform）

开发环境

Kotlin command-line compiler

手动安装 Kotlin 编译器：

- 下载最新版 (kotlin-compiler-2.2.20.zip)， 下载地址： GitHub Releases.
- 将独立编译器解压到一个目录中，并可选择将 bin 目录添加到系统路径。bin 目录包含在 Windows、macOS 和 Linux 上编译和运行 Kotlin 所需的脚本。

创建并运行应用程序

1. 使用 Kotlin 创建一个简单的控制台 JVM 应用程序，显示“Hello, World!”。在代码编辑器中，创建一个名为 hello.kt 的新文件，并写入以下代码：

```
fun main() {
    println("Hello, World!")
}
```

2. 使用 Kotlin 编译器编译应用程序：

```
kotlinc hello.kt -include-runtime -d hello.jar
```

- -d 选项表示生成的类文件的输出路径，可以是目录也可以是 .jar 文件。
- -include-runtime 选项通过在其中包含 Kotlin 运行时库，使生成的 .jar 文件自包含且可运行。

3. 运行生成的 JAR 文件：

```
java -jar hello.jar
输出：
Hello, World!
```

在线 编辑器

- Kotlin Playground

> https://play.kotlinlang.org/

思考：

Kotlin 与 Java 有什么区别和联系？

第二章 基本语法

Hello World

```
fun main() {
    println("Hello, Kotlin!")
}
```

变量与常量

```kotlin
val a: Int = 10   // 不可变
var b: String = "Kotlin" // 可变
```

- `val` 定义常量
- `var` 定义变量

类型推断

```
val x = 42          // 推断为 Int
var y = "Hello"     // 推断为 String
```

字符串模板

```
val name = "Tom"
println("Hello, $name!")
```

第三章 数据类型

- 基本类型：Int, Double, Float, Char, Boolean
- 字符串 String
- 数组 Array
- 集合 List, Set, Map

数组

```
val arr = arrayOf(1, 2, 3)
println(arr[0])
```

第四章 控制语句

- 条件语句：if, when
- 循环语句：for, while, do-while

if 表达式

```kotlin
val max = if (a > b) a else b
```

when 表达式

```
when(x) {
  1 -> println("One")
  2 -> println("Two")
  else -> println("Other")
}
```

> 思考：when 可以替代哪些 Java 语句？

for 循环

```
for (i in 1..5) {
    println(i)
}
```

while 循环

```
var i = 0
while (i < 5) {
    println(i)
    i++
}
```

第五章 函数与 Lambda

函数定义

```
fun add(x: Int, y: Int): Int {
    return x + y
}
```

单表达式函数

```
fun square(x: Int) = x * x
```

Lambda 表达式

```
val sum = {a: Int, b: Int -> a + b}
println(sum(2, 3))
```

高阶函数

```
fun apply(x: Int, f: (Int) -> Int): Int {
    return f(x)
}
println(apply(5, { it * it }))
```

第六章 面向对象编程

类与对象

```kotlin
class Person(val name: String, var age: Int)
val p = Person("Tom", 20)
```

继承

```kotlin
open class Animal
class Dog: Animal()
```

接口

```
interface Runnable {
    fun run()
}
```

数据类

```kotlin
data class User(val name: String, val age: Int)
```

什么是 data class

data class（数据类）用于表示值对象 / DTO / 简单模型。它的主要目标是减少样板代码：编译器会为主构造函数中声明的属性自动生成 equals/hashCode/toString/componentN/copy 等常用方法。

基本语法

```kotlin
data class User(val name: String, val age: Int)
```

> 要求：主构造函数至少有 1 个参数，参数通常要用 val 或 var 声明为属性（否则不是属性就不会参与生成的方法）。

编译器自动生成的成员（关键点）

对于 data class C(val a: A, val b: B)，编译器会生成：

1. equals(other: Any?): Boolean —— 基于主构造参数逐一比较（结构性相等）。
2. hashCode(): Int —— 基于这些参数计算哈希。
3. toString(): String —— 形如 C(a=..., b=...)。
4. component1(), component2(), ... —— 用于解构声明（顺序与主构造参数顺序相同）。
5. copy(a: A = this.a, b: B = this.b): C —— 以当前实例为模板复制并可修改某些参数（浅拷贝）。

简单示例与输出

```kotlin
data class Person(val name: String, val age: Int)

fun main() {
    val p1 = Person("Tom", 20)
    println(p1)                          // Person(name=Tom, age=20)
    val p2 = p1.copy(age = 21)
    println(p2)                          // Person(name=Tom, age=21)
    val (n, a) = p1
    println("$n, $a")                    // Tom, 20
    println(p1 == Person("Tom", 20)) // true
}
```

解构（Destructuring）

componentN 支持在赋值或 for 循环中直接解构：

```
val (x, y) = Point(1, 2)   // component1->x, component2->y
for ((k, v) in map) { ... } // Map.Entry 的 component1/component2
```

copy 的常用场景

创建略有不同的对象（不可变风格）

```
val u = User("Alice", 30)
val u2 = u.copy(age = u.age + 1)
```

copy() 是浅拷贝：对象引用类型字段仅复制引用（需要深拷贝时需自定义逻辑）。

主构造参数以外的属性不会参与生成的方法

```
data class Item(val id: Int) {
    var temp: String = "x"   // 不在 equals/hashCode/toString/copy 中
}
```

temp 不会出现在 toString、copy、equals、hashCode，常用于缓存、临时状态。

重要规则与限制

- data 类不能声明为 abstract, open, sealed 或 inner。
- 可以实现接口：data class D(...): SomeInterface
- 生成的 componentN、copy、toString、equals、hashCode 会基于主构造参数（顺序重要）。
- 如果你手动实现了某个方法（例如自定义 equals），编译器将不再生成该方法的默认实现。

常见注意事项

1. 可变字段（var）问题
   将 var 放在主构造函数中会使 hashCode/equals 基于可变值。如果把对象作为 Map 的 key 或 Set 的元素，修改该字段会破坏集合语义。

```
data class P(var name: String)
val p = P("A")
val map = mutableMapOf(p to "v")
p.name = "B"
println(map[p]) // 可能为 null —— 因为 hashCode/equals 已变
```

2. 数组比较问题

   如果主构造包含 Array 或原生数组 (IntArray 等)，equals 使用的是引用相等（Java 数组的 equals），导致内容相同的两个数组比较为 false。

```
data class D(val arr: IntArray)
val d1 = D(intArrayOf(1,2)); val d2 = D(intArrayOf(1,2))
println(d1 == d2) // false
// 解决: 用 List 或 自定义 equals 使用 contentEquals
```

建议用 List 替代数组做值语义，或手动实现 equals/hashCode。

3. 只包含主构造的属性参与比较
   如果你把某个重要字段放在类体中（不是主构造），它不会影响 equals/hashCode。这是常见的设计失误。

4. 二进制兼容性
   修改 data class 的主构造参数（增删字段）会改变生成的方法签名，可能导致二进制兼容性问题（对外库 API 要谨慎变更）。

自定义生成方法（覆盖默认实现）

如果默认生成的 equals/hashCode/toString/copy 不是所需，可手动实现它们：

```
data class Person(val id: Int, val name: String) {
    override fun equals(other: Any?) = (other is Person) && other.id == this.id
    override fun hashCode() = id.hashCode()
    // 编译器不再生成 equals/hashCode
}
```

> 但手动实现时要确保与 hashCode/equals 语义一致。

何时使用 data class（建议）

- 值对象（value objects）、DTO、接口返回值、消息体（message）等。
- 需要默认 equals/hashCode/toString 的场景。
- 不适合：JPA 实体或任何以“身份（identity）”而不是“值”为主的对象（例如数据库 ORM 实体通常需特殊对待）。

课堂练习 (进阶)

1. 写一个 data class Book(val title: String, val authors: List<String>, val year: Int)，演示 toString, copy, 与解构。
2. 证明 data class D(val arr: IntArray) 的两个实例即便数组内容相同也不相等，并修改类使之基于数组内容相等。
3. 说明为什么下面代码会导致 map[p] 返回 null:

```
data class P(var name: String)
val p = P("a");
val m = hashMapOf(p to 1)
p.name = "b"
println(m[p])
```

4. 把一个普通的 Java POJO 转换成 Kotlin data class，比较可读性和样板代码的减少。

参考答案（第1题）

```kotlin
data class Book(
    val title: String,
    val authors: List<String>,
    val year: Int
)

fun main() {
    // 创建 Book 对象
    val book1 = Book(
        "Kotlin 程序设计",
        listOf("张三", "李四"),
        2025
    )

    // 1. toString()
    println(book1)

    // 2. copy()
    val book2 = book1.copy(year = 2026)

    println(book2)

    // 3. 解构
    val (title, authors, year) = book1

    println("书名: $title")
    println("作者: $authors")
    println("出版年份: $year")
}
```

Output:

```
Book(title=Kotlin 程序设计, authors=[张三, 李四], year=2025)
Book(title=Kotlin 程序设计, authors=[张三, 李四], year=2026)
书名: Kotlin 程序设计
作者: [张三, 李四]
出版年份: 2025
```

参考答案（第2题）

```
data class D(val arr: IntArray)

fun main() {
    val d1 = D(intArrayOf(1, 2, 3))
    val d2 = D(intArrayOf(1, 2, 3))

    println(d1)
    println(d2)

    println(d1 == d2)
}

Output:
D(arr=[1, 2, 3])
D(arr=[1, 2, 3])
false
```

修改方法：自己实现 equals() 和 hashCode()

```
class D(val arr: IntArray) {

    override fun equals(other: Any?): Boolean {
        if (this === other) return true
        if (other !is D) return false

        return arr.contentEquals(other.arr)
    }

    override fun hashCode(): Int {
        return arr.contentHashCode()
    }

    override fun toString(): String {
        return "D(arr=${arr.contentToString()})"
    }
}

fun main() {
    val d1 = D(intArrayOf(1, 2, 3))
    val d2 = D(intArrayOf(1, 2, 3))

    println(d1)
    println(d2)

    println(d1 == d2)
}
```

参考答案（第3题）

关键在于：data class 自动生成的 hashCode() 会使用 name 属性。

创建对象：

```
val p = P("a")
```

此时：

```
name = "a"
```

假设：

```
hashCode(P("a")) = 100
```

执行：

```
val m = hashMapOf(p to 1)
```

此时 HashMap 内部存储了：

```
key: p (hashCode = 100) -> value: 1
```

执行：

```
p.name = "b"
```

由于 name 是 data class 的属性，hashCode() 也发生变化：

```
hashCode(P("b")) = 200
```

此时再执行：

```
println(m[p])
```

HashMap 会根据当前的 HashCode 去寻找：

```
key: p (hashCode = 200)
```

但是 HashMap 内部存储的 key 是：

```
key: p (hashCode = 100)
```

因此找不到对应的 entry，返回 null。

参考答案（第4题）

```
public class Book {

    private String title;
    private String author;
    private int year;

    public Book(String title, String author, int year) {
        this.title = title;
        this.author = author;
        this.year = year;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public int getYear() {
        return year;
    }

    public void setYear(int year) {
        this.year = year;
    }

    @Override
    public String toString() {
        return "Book{" +
                "title='" + title + '\'' +
                ", author='" + author + '\'' +
                ", year=" + year +
                '}';
    }

    @Override
    public boolean equals(Object o) {
        // equals implementation
        return true;
    }

    @Override
    public int hashCode() {
        // hashCode implementation
        return 0;
    }
}
```

```kotlin
data class Book(
    val title: String,
    val author: String,
    val year: Int
)
```

第七章 集合与泛型

集合

```kotlin
val list = listOf(1, 2, 3)
val map = mapOf("a" to 1, "b" to 2)
```

集合操作

```kotlin
val result = list.filter { it > 1 }.map { it * 2 }
```

泛型函数

```
fun <T> singletonList(item: T): List<T> {
    return listOf(item)
}
```

第八章 空安全

- Kotlin 默认变量不可为 null
- 使用 `?` 声明可空类型

```
var str: String? = null
println(str?.length)
```

Kotlin 的空安全（Null Safety）

- Kotlin 特性：**在编译期防止空指针异常（NPE）**
- 类型系统区分：
  - **非空类型**：`String` → 不允许为 `null`
  - **可空类型**：`String?` → 变量可能为 `null`

安全调用（?.）

- 使用 `?.` 安全访问可空对象属性或方法
- 如果对象为 `null`，表达式直接返回 `null`

```
var b: String? = "Hi"
println(b?.length)  // 输出 2
b = null
println(b?.length)  // 输出 null
```

Elvis 操作符

- 为可空表达式提供默认值
- 常用于避免 null 传播

```
val b: String? = null
val len = b?.length ?: 0
println(len)   // 输出 0
```

非空断言

- 强制把可空类型当作非空使用
- 若实际为 null → 抛出 NPE（不推荐滥用）

```
val b: String? = null
val len = b!!.length   // 运行时抛出 NullPointerException
```

第九章 扩展函数

```kotlin
fun String.lastChar(): Char = this[this.length - 1]
```

第十章 协程

基本示例

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        delay(1000L)
        println("World!")
    }
    println("Hello,")
}
```

第十一章 Kotlin 与 Java 互操作

- 直接调用 Java 类库
- Java 代码调用 Kotlin 类

Java 调用 Kotlin

```java
User user = new User("Tom", 20);
```

Kotlin 调用 Java

```kotlin
val list = ArrayList<String>()
list.add("Hello")
```

第十二章 实践案例

Android 示例

```
class MainActivity: AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

案例：Web 后端

```
fun main() {
    embeddedServer(Netty, port = 8080) {
        routing {
            get("/") {
                call.respondText("Hello, Kotlin Server!")
            }
        }
    }.start(wait = true)
}
```

总结

- Kotlin 是现代化语言，简洁安全
- 强大的空安全和协程支持
- 广泛应用于 Android、后端、跨平台
- 与 Java 无缝互操作

参考资料

- Kotlin 官方文档
- 《Kotlin 实战》
- 《Kotlin 编程权威指南》